17.2 工作区与上下文文件注入

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~450,000 tokens，输出 ~36,000 tokens（本章合计）

在 9.1 节中，我们看到 `buildAgentSystemPrompt` 函数会将一组 `EmbeddedContextFile` 注入到系统提示词的 `# Project Context` 段落中。但这些文件从哪来？它们是怎么被加载、过滤、截断，最终变成提示词的一部分的？

本节追踪上下文文件的完整生命周期：从工作区磁盘上的 Markdown 文件，到最终嵌入 LLM 上下文窗口的文本片段。

17.2.1 工作区根目录解析（`src/agents/workspace.ts`）

默认工作区路径

每个 OpenClaw 实例都有一个**工作区目录**，存放 Agent 运行所需的引导文件和用户配置。默认路径为：

```
// src/agents/workspace.ts
export function resolveDefaultAgentWorkspaceDir(
  env: NodeJS.ProcessEnv = process.env,
  homedir: () => string = os.homedir,
): string {
  const profile = env.OPENCLAW_PROFILE?.trim();
  if (profile && profile.toLowerCase() !== "default") {
    return path.join(homedir(), ".openclaw", `workspace-${profile}`);
  }
  return path.join(homedir(), ".openclaw", "workspace");
}
```

- **默认 Profile**：`~/.openclaw/workspace`
- **自定义 Profile**：`~/.openclaw/workspace-{profile}`

多 Profile 支持允许在同一台机器上运行多个 OpenClaw 实例，每个实例有独立的工作区。

> **衍生解释——Profile（配置档）**
>
> Profile 是一种常见的软件配置隔离模式。类似于浏览器的"用户档案"或 AWS CLI 的 `--profile` 参数，OpenClaw 的 Profile 通过环境变量 `OPENCLAW_PROFILE` 控制，使得同一用户可以维护多套完全独立的 Agent 配置。

八大引导文件

OpenClaw 定义了八个标准的**引导文件（Bootstrap Files）**，构成了 Agent 的"知识库"：

```
// src/agents/workspace.ts
export const DEFAULT_AGENTS_FILENAME = "AGENTS.md";
export const DEFAULT_SOUL_FILENAME = "SOUL.md";
export const DEFAULT_TOOLS_FILENAME = "TOOLS.md";
export const DEFAULT_IDENTITY_FILENAME = "IDENTITY.md";
export const DEFAULT_USER_FILENAME = "USER.md";
export const DEFAULT_HEARTBEAT_FILENAME = "HEARTBEAT.md";
export const DEFAULT_BOOTSTRAP_FILENAME = "BOOTSTRAP.md";
export const DEFAULT_MEMORY_FILENAME = "MEMORY.md";
```

文件	用途	对应 PromptMode
`AGENTS.md`	Agent 行为指导、项目规范	full + minimal
`SOUL.md`	人格定义、语调、风格	full
`TOOLS.md`	工具使用指南（用户自定义）	full + minimal
`IDENTITY.md`	Agent 身份信息（名字、emoji、头像）	full
`USER.md`	用户偏好信息	full
`HEARTBEAT.md`	心跳检查的自定义提示	full
`BOOTSTRAP.md`	通用引导信息	full
`MEMORY.md`	长期记忆存储	full

`AGENTS.md` 和 `TOOLS.md` 是**唯二**会传递给子 Agent 的引导文件（见下文的 `SUBAGENT_BOOTSTRAP_ALLOWLIST`）。

工作区初始化

`ensureAgentWorkspace` 负责创建工作区目录并填充默认的引导文件模板：

```
// src/agents/workspace.ts（简化）
export async function ensureAgentWorkspace(params?: {
  dir?: string;
  ensureBootstrapFiles?: boolean;
}): Promise<{ dir: string; agentsPath?: string; ... }> {
  const dir = resolveUserPath(rawDir);
  await fs.mkdir(dir, { recursive: true });

  if (!params?.ensureBootstrapFiles) return { dir };

  // 检查是否为全新工作区（所有文件都不存在）
  const isBrandNewWorkspace = await (async () => {
    const paths = [agentsPath, soulPath, toolsPath, ...];
    const existing = await Promise.all(
      paths.map(async (p) => {
        try { await fs.access(p); return true; }
        catch { return false; }
      })
    );
    return existing.every((v) => !v);
  })();

  // 从模板目录加载默认内容
  const agentsTemplate = await loadTemplate(DEFAULT_AGENTS_FILENAME);
  const soulTemplate = await loadTemplate(DEFAULT_SOUL_FILENAME);
  // ...加载其他模板

  // 仅在文件不存在时写入（不覆盖用户修改）
  await writeFileIfMissing(agentsPath, agentsTemplate);
  await writeFileIfMissing(soulPath, soulTemplate);
  // ...写入其他文件

  // 全新工作区还会初始化 Git 仓库
  await ensureGitRepo(dir, isBrandNewWorkspace);

  return { dir, agentsPath, soulPath, ... };
}
```

几个关键设计决策：

1. **`writeFileIfMissing`** 使用 `fs.writeFile` 的 `wx` 标志（排他创建），确保不会覆盖用户已有的文件。
2. **全新工作区检测**：只有当所有标准文件都不存在时，才认为是全新工作区，此时才会写入 `BOOTSTRAP.md` 并初始化 Git。
3. **Git 初始化**：自动 `git init` 的目的是让工作区的文件变更可追踪，但 Git 不可用时不会报错。

Front Matter 剥离

模板文件可能包含 YAML Front Matter（用于文档系统的元数据），加载时需要剥离：

```
function stripFrontMatter(content: string): string {
  if (!content.startsWith("---")) return content;
  const endIndex = content.indexOf("\n---", 3);
  if (endIndex === -1) return content;
  const start = endIndex + "\n---".length;
  return content.slice(start).replace(/^\s+/, "");
}
```

> **衍生解释——YAML Front Matter**
>
> YAML Front Matter 是 Markdown 文件开头以 `---` 包裹的 YAML 格式元数据块，常见于 Jekyll、Hugo 等静态站点生成器。例如：
>
> ```
> ---
> title: AGENTS.md Template
> version: 2
> ---
> # Actual Content...
> ```
>
> OpenClaw 的模板文件使用 Front Matter 存储版本信息，但注入到提示词时需要移除这些元数据。

引导文件加载

`loadWorkspaceBootstrapFiles` 一次性加载所有引导文件：

```typescript
// src/agents/workspace.ts
export async function loadWorkspaceBootstrapFiles(
  dir: string
): Promise<WorkspaceBootstrapFile[]> {
  const resolvedDir = resolveUserPath(dir);

  // 固定的文件列表
  const entries = [
    { name: DEFAULT_AGENTS_FILENAME, filePath: path.join(resolvedDir, "AGENTS.md") },
    { name: DEFAULT_SOUL_FILENAME,   filePath: path.join(resolvedDir, "SOUL.md") },
    { name: DEFAULT_TOOLS_FILENAME,  filePath: path.join(resolvedDir, "TOOLS.md") },
    { name: DEFAULT_IDENTITY_FILENAME, filePath: path.join(resolvedDir, "IDENTITY.md")
    { name: DEFAULT_USER_FILENAME,   filePath: path.join(resolvedDir, "USER.md") },
    { name: DEFAULT_HEARTBEAT_FILENAME, filePath: path.join(resolvedDir, "HEARTBEAT.md"
    { name: DEFAULT_BOOTSTRAP_FILENAME, filePath: path.join(resolvedDir, "BOOTSTRAP.md"
  ];

  // 追加 MEMORY.md（可选，有去重逻辑）
  entries.push(...(await resolveMemoryBootstrapEntries(resolvedDir)));

  // 逐个读取，缺失则标记 missing
  const result: WorkspaceBootstrapFile[] = [];
  for (const entry of entries) {
    try {
      const content = await fs.readFile(entry.filePath, "utf-8");
      result.push({ name: entry.name, path: entry.filePath, content, missing: false });
    } catch {
      result.push({ name: entry.name, path: entry.filePath, missing: true });
    }
  }
  return result;
}
```

`resolveMemoryBootstrapEntries` 同时检查 `MEMORY.md` 和 `memory.md`（大小写变体），并使用 `fs.realpath` 进行去重——如果两者指向同一个文件（如通过符号链接），只保留一份。

子 Agent 文件过滤

子 Agent 只能接收 `AGENTS.md` 和 `TOOLS.md`，其他引导文件被过滤掉：

```
const SUBAGENT_BOOTSTRAP_ALLOWLIST = new Set([
  DEFAULT_AGENTS_FILENAME,
  DEFAULT_TOOLS_FILENAME,
]);

export function filterBootstrapFilesForSession(
  files: WorkspaceBootstrapFile[],
  sessionKey?: string,
): WorkspaceBootstrapFile[] {
  if (!sessionKey || !isSubagentSessionKey(sessionKey)) return files;
  return files.filter((file) => SUBAGENT_BOOTSTRAP_ALLOWLIST.has(file.name));
}
```

`isSubagentSessionKey` 通过 session key 的格式判断是否为子 Agent。这意味着子 Agent 不会获得 SOUL.md（人格）、USER.md（用户偏好）等信息——它们只需要知道项目规范和工具使用指南就够了。

17.2.2 引导文件（Bootstrap Files）：AGENTS.md / SOUL.md / TOOLS.md

AGENTS.md：项目规范文件

`AGENTS.md` 是最重要的引导文件，定义了 Agent 在特定项目中的行为规范。一个典型的 `AGENTS.md` 可能包含：

```
# AGENTS.md - AI Agent Operational Guidelines

## Project Structure
- src/ contains the main application code
- tests/ contains unit and integration tests

## Coding Standards
- Use TypeScript strict mode
- All functions must have JSDoc comments
- Tests must use vitest

## Deployment
- CI runs on GitHub Actions
- Deploy to AWS via CDK
```

AGENTS.md 的设计理念是让用户以自然语言描述项目的约束和规范，Agent 在执行任何操作前会参考这些指导。

SOUL.md：人格定义文件

`SOUL.md` 控制 Agent 的"人格"——语调、表达风格、角色设定。回顾 9.1 节，系统提示词中如果检测到 SOUL.md 存在，会插入特殊指令：

```
If SOUL.md is present, embody its persona and tone.
Avoid stiff, generic replies; follow its guidance unless
higher-priority instructions override it.
```

一个示例 SOUL.md：

```md
# Soul

You are "Aria", a friendly but precise engineering assistant.
- Keep responses concise and technical
- Use emoji sparingly (only for emphasis)
- When uncertain, say so directly
- Prefer code examples over verbose explanations
```

TOOLS.md：工具使用指南

`TOOLS.md` 是用户自定义的工具使用指南。注意它与系统提示词中的 Tooling 段落不同——Tooling 段落由 OpenClaw 自动生成，而 TOOLS.md 是用户编写的补充说明。系统提示词中明确标注：

```
TOOLS.md does not control tool availability;
it is user guidance for how to use external tools.
```

resolveBootstrapFilesForRun：引导文件的完整解析链

`resolveBootstrapFilesForRun`（位于 `bootstrap-files.ts`）将上述所有步骤串联为一个完整的解析链：

```typescript
// src/agents/bootstrap-files.ts
export async function resolveBootstrapFilesForRun(params: {
  workspaceDir: string;
  config?: OpenClawConfig;
  sessionKey?: string;
  sessionId?: string;
  agentId?: string;
}): Promise<WorkspaceBootstrapFile[]> {
  const sessionKey = params.sessionKey ?? params.sessionId;

  // 步骤 1：加载所有引导文件
  const allFiles = await loadWorkspaceBootstrapFiles(params.workspaceDir);

  // 步骤 2：按会话类型过滤（子 Agent 只保留白名单文件）
  const filteredFiles = filterBootstrapFilesForSession(allFiles, sessionKey);

  // 步骤 3：应用引导钩子覆盖
  return applyBootstrapHookOverrides({
    files: filteredFiles,
    workspaceDir: params.workspaceDir,
    config: params.config,
    sessionKey: params.sessionKey,
    sessionId: params.sessionId,
    agentId: params.agentId,
  });
}
```

流程图：

```
磁盘文件                    过滤                    钩子

┌──────────┐          ┌──────────┐          ┌──────────┐
│ AGENTS.md│          │ 子Agent? │          │ Hook     │
│ SOUL.md  │  ──────► │ 白名单过滤 │  ──────► │ 修改/替换 │  ──────►  最终列表
│ TOOLS.md │          │          │          │          │
│ ...      │          └──────────┘          └──────────┘
└──────────┘
```

resolveBootstrapContextForRun：文件 → 上下文片段

最终一步是将 `WorkspaceBootstrapFile[]` 转换为 `EmbeddedContextFile[]`（可直接注入提示词的格式）：

```ts
// src/agents/bootstrap-files.ts
export async function resolveBootstrapContextForRun(params: {
  workspaceDir: string;
  config?: OpenClawConfig;
  sessionKey?: string;
  sessionId?: string;
  agentId?: string;
  warn?: (message: string) => void;
}): Promise<{
  bootstrapFiles: WorkspaceBootstrapFile[];
  contextFiles: EmbeddedContextFile[];
}> {
  const bootstrapFiles = await resolveBootstrapFilesForRun(params);
  const contextFiles = buildBootstrapContextFiles(bootstrapFiles, {
    maxChars: resolveBootstrapMaxChars(params.config),
    warn: params.warn,
  });
  return { bootstrapFiles, contextFiles };
}
```

截断策略

当引导文件内容超过最大字符限制（默认 20,000 字符）时， `buildBootstrapContextFiles` 会调用 `trimBootstrapContent` 进行截断：

```ts
// src/agents/pi-embedded-helpers/bootstrap.ts
const DEFAULT_BOOTSTRAP_MAX_CHARS = 20_000;
const BOOTSTRAP_HEAD_RATIO = 0.7;  // 保留前 70%
const BOOTSTRAP_TAIL_RATIO = 0.2;  // 保留后 20%

function trimBootstrapContent(
  content: string,
  fileName: string,
  maxChars: number,
): TrimBootstrapResult {
  const trimmed = content.trimEnd();
  if (trimmed.length <= maxChars) {
    return { content: trimmed, truncated: false, ... };
  }

  const headChars = Math.floor(maxChars * BOOTSTRAP_HEAD_RATIO);
  const tailChars = Math.floor(maxChars * BOOTSTRAP_TAIL_RATIO);
  const head = trimmed.slice(0, headChars);
  const tail = trimmed.slice(-tailChars);

  const marker = [
    "",
    `[...truncated, read ${fileName} for full content...]`,
    `…(truncated ${fileName}: kept ${headChars}+${tailChars} chars of ${trimmed.length}
    "",
  ].join("\n");

  return {
    content: [head, marker, tail].join("\n"),
    truncated: true,
    ...
  };
}
```

截断策略采用了"保头保尾"的设计：

- **前 70%**：文件开头通常包含最重要的信息（标题、核心规则）
- **后 20%**：文件结尾可能包含最近添加的条目或总结
- **中间 10%**：作为截断标记的空间，告知模型可以用 `read` 工具阅读完整内容

对于缺失的文件，会注入一个标记：

```
if (file.missing) {
  result.push({
    path: file.name,
    content: `[MISSING] Expected at: ${file.path}`,
  });
  continue;
}
```

17.2.3 引导钩子（Bootstrap Hooks）（ src/agents/bootstrap-hooks.ts ）

引导钩子允许外部插件在引导文件加载后、注入提示词前**修改或替换**文件内容。

钩子机制

```
// src/agents/bootstrap-hooks.ts
export async function applyBootstrapHookOverrides(params: {
  files: WorkspaceBootstrapFile[];
  workspaceDir: string;
  config?: OpenClawConfig;
  sessionKey?: string;
  sessionId?: string;
  agentId?: string;
}): Promise<WorkspaceBootstrapFile[]> {
  const agentId = params.agentId ??
    (params.sessionKey
      ? resolveAgentIdFromSessionKey(params.sessionKey)
      : undefined);

  // 构建钩子上下文
  const context: AgentBootstrapHookContext = {
    workspaceDir: params.workspaceDir,
    bootstrapFiles: params.files,
    cfg: params.config,
    sessionKey: params.sessionKey,
    sessionId: params.sessionId,
    agentId,
  };

  // 创建并触发内部钩子事件
  const event = createInternalHookEvent(
    "agent",          // 命名空间
    "bootstrap",      // 事件名
    sessionKey,       // 事件源
    context,          // 可变上下文
  );
  await triggerInternalHook(event);

  // 钩子可以修改 context.bootstrapFiles
  const updated = (event.context as AgentBootstrapHookContext).bootstrapFiles;
  return Array.isArray(updated) ? updated : params.files;
}
```

钩子的工作方式是**原地修改上下文对象**：钩子处理器接收到 `context.bootstrapFiles` 数组后，可以：

- **追加**新的引导文件
- **移除**不需要的文件
- **替换**文件内容
- **重新排序**文件

如果钩子没有修改 `bootstrapFiles` （或返回了非数组值），则使用原始文件列表作为回退。

> **衍生解释——Hook（钩子）模式**
>
> 钩子是一种扩展点模式，允许外部代码在特定事件发生时"挂入"并修改行为。在 OpenClaw 中， `triggerInternalHook` 会遍历所有注册的钩子处理器，按顺序调用它们。这类似于 Webpack 的 Plugin 系统——插件通过监听特定的钩子事件来扩展核心功能，而无需修改核心代码。

使用场景

引导钩子的典型使用场景包括：

1. **条件性注入**：根据当前 Agent ID 或通道，动态添加特定的引导内容。
2. **敏感信息过滤**：在将文件注入提示词前，移除不应暴露给特定会话的内容。
3. **内容增强**：从外部数据源拉取信息并注入到引导文件中。

17.2.4 工作区模板（ `src/agents/workspace-templates.ts` ）

模板目录解析

工作区首次初始化时，需要从模板目录加载默认的引导文件内容。 `resolveWorkspaceTemplateDir` 负责找到模板目录的位置：

```typescript
// src/agents/workspace-templates.ts
const FALLBACK_TEMPLATE_DIR = path.resolve(
  path.dirname(fileURLToPath(import.meta.url)),
  "../../docs/reference/templates",
);

let cachedTemplateDir: string | undefined;

export async function resolveWorkspaceTemplateDir(opts?: {
  cwd?: string;
  argv1?: string;
  moduleUrl?: string;
}): Promise<string> {
  if (cachedTemplateDir) return cachedTemplateDir;

  const packageRoot = await resolveOpenClawPackageRoot({ ... });
  const candidates = [
    packageRoot ? path.join(packageRoot, "docs", "reference", "templates") : null,
    cwd ? path.resolve(cwd, "docs", "reference", "templates") : null,
    FALLBACK_TEMPLATE_DIR,
  ].filter(Boolean) as string[];

  for (const candidate of candidates) {
    if (await pathExists(candidate)) {
      cachedTemplateDir = candidate;
      return candidate;
    }
  }

  cachedTemplateDir = candidates[0] ?? FALLBACK_TEMPLATE_DIR;
  return cachedTemplateDir;
}
```

解析策略按优先级排列：

1. **包根目录**下的 `docs/reference/templates/` （npm 安装场景）
2. **当前工作目录**下的 `docs/reference/templates/` （开发场景）
3. **回退路径**：相对于当前模块的 `../../docs/reference/templates`

缓存策略

模板目录路径使用**模块级缓存**（`cachedTemplateDir`），确保整个进程生命周期内只解析一次。`resetWorkspaceTemplateDirCache` 函数用于测试时重置缓存：

```
export function resetWorkspaceTemplateDirCache() {
  cachedTemplateDir = undefined;
  resolvingTemplateDir = undefined;
}
```

并发安全

`resolveWorkspaceTemplateDir` 使用了 `resolvingTemplateDir` Promise 变量来防止并发解析。当多个调用同时到达时，第一个调用开始实际解析，后续调用等待同一个 Promise：

```
let resolvingTemplateDir: Promise<string> | undefined;

export async function resolveWorkspaceTemplateDir(...): Promise<string> {
  if (cachedTemplateDir) return cachedTemplateDir;
  if (resolvingTemplateDir) return resolvingTemplateDir; // 等待正在进行的解析

  resolvingTemplateDir = (async () => {
    // 实际的解析逻辑...
  })();

  try {
    return await resolvingTemplateDir;
  } finally {
    resolvingTemplateDir = undefined; // 重置，允许下次重新解析（如果缓存被清除）
  }
}
```

> **衍生解释——Promise 去重（Deduplication）**
>
> 这是一种常见的异步编程模式：当多个调用者请求同一个异步资源时，只实际执行一次操作，所有调用者共享同一个 Promise。这避免了重复的文件系统操作或网络请求。在 OpenClaw 中，这种模式用于模板目录解析、模型配置加载等多处场景。

完整的上下文注入流程

把本节所有内容串联起来，从磁盘文件到最终注入提示词的完整流程如下：

```
┌──────────────────────────────────────────────────────────┐
│ 1. resolveWorkspaceTemplateDir                           │
│    找到模板目录 → docs/reference/templates/              │
└──────────────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────────────┐
│ 2. ensureAgentWorkspace                                  │
│    创建工作区 → 从模板初始化引导文件（writeFileIfMissing）│
└──────────────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────────────┐
│ 3. loadWorkspaceBootstrapFiles                           │
│    从工作区加载 8 个引导文件 → WorkspaceBootstrapFile[]  │
└──────────────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────────────┐
│ 4. filterBootstrapFilesForSession                        │
│    子 Agent 过滤 → 仅保留 AGENTS.md + TOOLS.md           │
└──────────────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────────────┐
│ 5. applyBootstrapHookOverrides                           │
│    触发 "agent:bootstrap" 钩子 → 允许插件修改文件列表    │
└──────────────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────────────┐
│ 6. buildBootstrapContextFiles                            │
│    截断超长文件（保头 70% + 尾 20%）→ EmbeddedContextFile│
└──────────────────────────────────────────────────────────┘
                      │
                      ▼
┌──────────────────────────────────────────────────────────┐
│ 7. buildAgentSystemPrompt                                │
│    将 EmbeddedContextFile[] 注入 # Project Context 段落  │
└──────────────────────────────────────────────────────────┘
```

本节小结

1. **工作区**（Workspace）是 OpenClaw 的核心文件系统抽象，默认位于 `~/.openclaw/workspace`，支持通过 Profile 机制实现多实例隔离。

2. **八大引导文件**各有分工： `AGENTS.md` 定义项目规范， `SOUL.md` 定义人格， `TOOLS.md` 指导工具使用， `IDENTITY.md` 承载身份信息，等等。
3. **子 Agent 过滤**确保子 Agent 只获得必要的上下文（AGENTS.md 和 TOOLS.md），避免注入无关信息。
4. **引导钩子**提供了扩展点，允许插件在文件加载后、注入前修改引导文件内容。
5. **模板系统**为新工作区提供开箱即用的默认配置，使用"只写不覆盖"策略保护用户已有的自定义内容。
6. **截断策略**（保头 70% + 尾 20%）在控制 token 消耗的同时，尽量保留文件中最重要的信息。